



Apex Institute of Technology

Computer Science & Engineering

Experiment 7

Name: Shivam Abhepal

UID: 24BAI70214

Branch: B.E. CSE (AIML)

Section: 24AIT_KRG-G1

Semester: 4

Date of Performance: 25.03.2026

Subject Name: Database
Management System

Subject Code: 24CSH-298

AIM: This experiment focuses on designing and implementing simple views, complex views, and materialized views in a database system. It demonstrates how materialized views improve query performance by storing precomputed results and compares their efficiency with regular views.

OBJECTIVES:

- To understand the concept of views and materialized views.
- To differentiate between simple views, complex views, and materialized views.
- To create and implement views in SQL.
- To analyze performance differences between views.
- To understand the importance of materialized views in real-world applications.

SOFTWARE REQUIREMENTS:

- Oracle FreeSQL / Oracle Database Express Edition (Oracle XE)
- Oracle SQL Developer
- PostgreSQL (pgAdmin / DB Fiddle)

PROCEDURE/STEPS:

1. Start the system and log in.
2. Open Oracle SQL Developer / DB Fiddle.



Apex Institute of Technology

Computer Science & Engineering

3. Connect to the required database.
4. Open SQL Worksheet / Query Tool.
5. Create employee and department tables.
6. Insert sample data into tables.
7. Create simple and complex views.
8. Create a materialized view.
9. Execute queries and observe results.
10. Compare performance and record observations.

PRACTICAL/EXPERIMENTAL STEPS:

Table Creation

```
-- Drop tables (ignore error if not exists)
DROP TABLE employees CASCADE CONSTRAINTS;
DROP TABLE departments CASCADE CONSTRAINTS;
```

```
-- Create tables
CREATE TABLE departments (
    department_id NUMBER PRIMARY KEY,
    department_name VARCHAR2(100)
);
```

```
CREATE TABLE employees (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100),
    department_id NUMBER,
    salary NUMBER(10,2),
    hire_date DATE
);
```

```
-- Insert data
INSERT INTO departments VALUES (1, 'HR');
INSERT INTO departments VALUES (2, 'IT');
INSERT INTO departments VALUES (3, 'Finance');
INSERT INTO departments VALUES (4, 'Marketing');
```

```
INSERT INTO employees VALUES (1, 'Amit', 1, 40000, DATE '2022-01-10');
INSERT INTO employees VALUES (2, 'Ravi', 2, 60000, DATE '2021-03-15');
```



Apex Institute of Technology

Computer Science & Engineering

```
INSERT INTO employees VALUES (3, 'Priya', 2, 75000, DATE '2020-07-20');  
INSERT INTO employees VALUES (4, 'Neha', 3, 50000, DATE '2022-05-11');  
  
COMMIT;
```

Simple View

```
CREATE VIEW simple_employee_view AS  
SELECT emp_id, emp_name, salary  
FROM employees;  
SELECT * FROM simple_employee_view;
```

Complex View

```
CREATE VIEW complex_employee_view AS  
SELECT  
    d.department_name,  
    COUNT(e.emp_id) AS total_employees,  
    AVG(e.salary) AS avg_salary  
FROM employees e  
JOIN departments d  
ON e.department_id = d.department_id  
GROUP BY d.department_name;  
SELECT * FROM complex_employee_view;
```

Materialized View

```
CREATE MATERIALIZED VIEW materialized_employee_view  
BUILD IMMEDIATE  
REFRESH COMPLETE  
AS  
SELECT  
    d.department_name,  
    COUNT(e.emp_id) AS total_employees,  
    AVG(e.salary) AS avg_salary  
FROM employees e  
JOIN departments d
```



Apex Institute of Technology

Computer Science & Engineering

```
ON e.department_id = d.department_id  
GROUP BY d.department_name;  
SELECT * FROM materialized_employee_view;
```

OUTPUT:(Screenshots)

Simple View

The screenshot shows an SQL worksheet with the following SQL code:

```
16 INSERT INTO departments VALUES (3, 'Finance');  
17 INSERT INTO departments VALUES (4, 'Marketing');  
18  
19 INSERT INTO employees VALUES (1, 'Anhad', 1, 40000, DATE '2022-01-10');  
20 INSERT INTO employees VALUES (2, 'Rohit', 2, 60000, DATE '2021-03-15');  
21 INSERT INTO employees VALUES (3, 'Jaskaran', 2, 75000, DATE '2020-07-20');  
22 INSERT INTO employees VALUES (4, 'Yuvraj', 3, 50000, DATE '2022-05-11');  
23  
24 COMMIT;  
25  
26 CREATE VIEW simple_employee_view AS  
27 SELECT emp_id, emp_name, salary  
28 FROM employees;  
29 SELECT * FROM simple_employee_view;
```

Below the code editor, the 'Query result' tab is active, showing the following table:

	EMP_ID	EMP_NAME	SALARY
1	1	Anhad	40000
2	2	Rohit	60000
3	3	Jaskaran	75000
4	4	Yuvraj	50000



Apex Institute of Technology

Computer Science & Engineering

Complex View

[SQL Worksheet]*

```
22 INSERT INTO employees VALUES (4, 'Yuvraj', 3, 50000, DATE '2022-05-11');
23
24 COMMIT;
25
26 CREATE VIEW complex_employee_view AS
27 SELECT
28     d.department_name,
29     COUNT(e.emp_id) AS total_employees,
30     AVG(e.salary) AS avg_salary
31 FROM employees e
32 JOIN departments d
33 ON e.department_id = d.department_id
34 GROUP BY d.department_name;
35 SELECT * FROM complex_employee_view;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download Execution time: 0.014 seconds

	DEPARTMENT_NAME	TOTAL_EMPLOYEES	AVG_SALARY
1	HR	1	40000
2	IT	2	67500
3	Finance	1	50000

Materialized View

[SQL Worksheet]*

```
21 INSERT INTO employees VALUES (3, 'Saksham', 2, 75000, DATE '2020-07-20');
22 INSERT INTO employees VALUES (4, 'Yuvraj', 3, 50000, DATE '2022-05-11');
23
24 COMMIT;
25
26 CREATE MATERIALIZED VIEW materialized_employee_view
27 BUILD IMMEDIATE
28 REFRESH COMPLETE
29 AS
30 SELECT
31     d.department_name,
32     COUNT(e.emp_id) AS total_employees,
33     AVG(e.salary) AS avg_salary
34 FROM employees e
35 JOIN departments d
36 ON e.department_id = d.department_id
37 GROUP BY d.department_name;
38 SELECT * FROM materialized_employee_view;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download Execution time: 0.006 seconds

	DEPARTMENT_NAME	TOTAL_EMPLOYEES	AVG_SALARY
1	HR	1	40000
2	IT	2	67500
3	Finance	1	50000



Apex Institute of Technology

Computer Science & Engineering

I/O ANALYSIS:

Input:

- Employee ID
- Employee Name
- Department ID
- Salary
- Hire Date

Output:

- Employee details using simple view
- Aggregated department data using complex view
- Precomputed results using materialized view

LEARNING OUTCOMES:

- Understood the concept of views and materialized views.
- Learned differences between simple and complex views.
- Implemented materialized views in Oracle/PostgreSQL.
- Analyzed performance improvement using materialized views.
- Gained knowledge of real-world database optimization techniques.

RESULT:

Materialized views store precomputed query results and significantly improve performance compared to complex views, especially for repeated query execution in large-scale systems.

CONCLUSION:

Materialized views are an effective optimization technique in database systems. They reduce query execution time and enhance system performance, making them highly useful in enterprise-level applications.